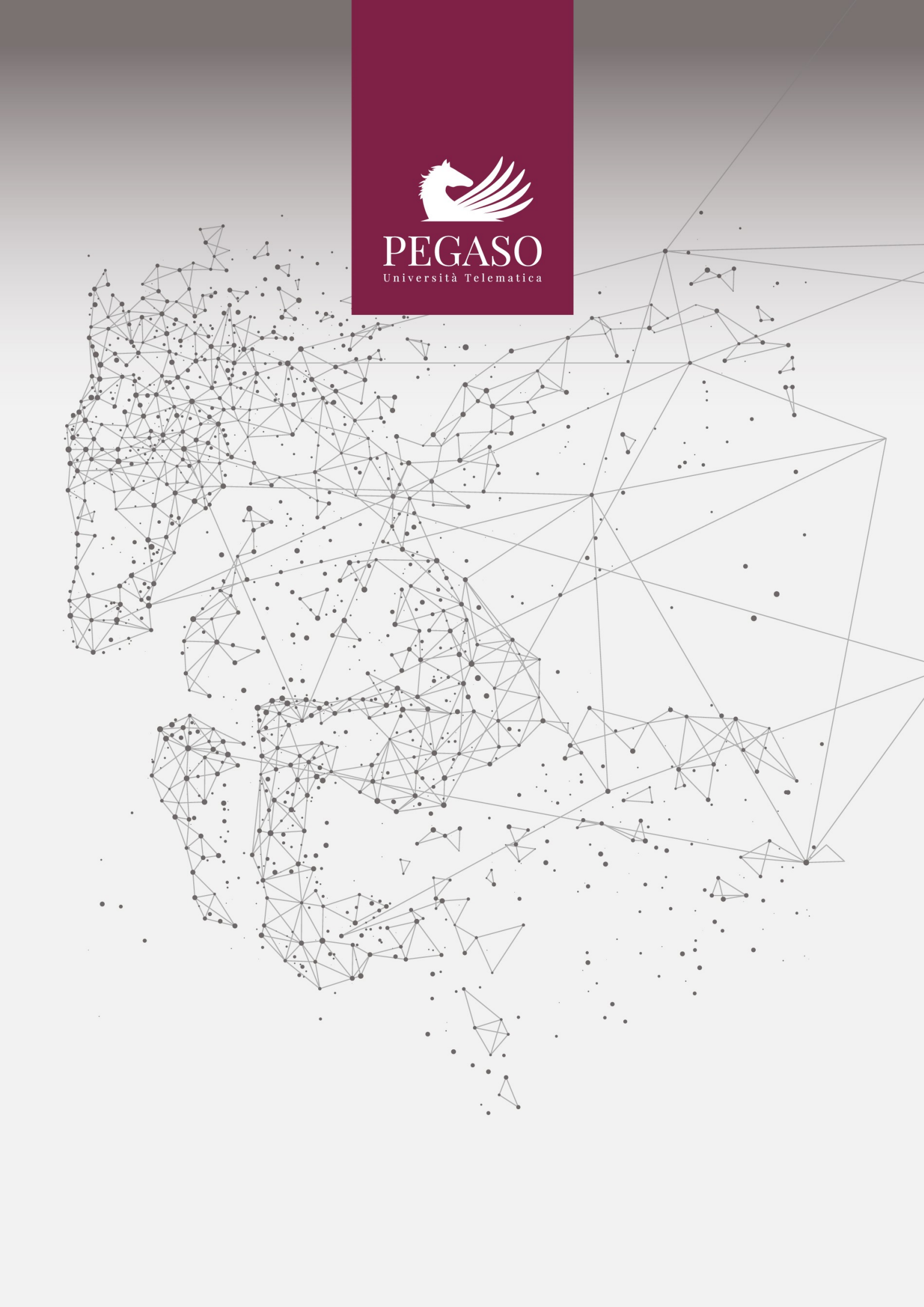

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. CLOCK LOGICI DI LAMPORT....4
3. VECTOR CLOCK....7
BIBLIOGRAFIA....10

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

È naturale pensare che la sincronizzazione del clock sia strettamente collegata al tempo, ma è possibile che non sia necessario avere un resoconto accurato del tempo reale. Può essere sufficiente che ogni nodo in un sistema distribuito concordi su un tempo corrente piuttosto che avere il resoconto accurato del tempo reale.

Siamo in grado di fare un ulteriore passo avanti a questo punto. Ai fini dell'esecuzione dell'operazione *make* per compilare dei file sorgente, è sufficiente che due nodi condividano l'informazione che, ad esempio, una versione precedente di *input.o* sia stata resa obsoleta da una versione più recente di *input.c*. In questo scenario, ciò che è importante è che entrambe le parti monitorino e tengano traccia degli eventi reciproci, ad esempio la produzione di una nuova versione di *input.c*. Quando si discute di questo tipo di algoritmi, è pratica comune riferirsi ai clock come "clock logici".

Lamport nel 1978 dimostrò in un articolo fondamentale che anche se la sincronizzazione del clock è possibile, non è necessario che sia assolutamente accurata. Non è necessario sincronizzare i clock di due processi se i processi stessi non interagiscono tra loro. Questo perché la mancanza di sincronizzazione tra i clock non sarebbe effettivamente osservabile e quindi non sarebbe in grado di causare alcun problema.

Oltre a questo, ha sollevato il punto che in genere non è importante che tutti i processi siano in accordo sul momento preciso, ma piuttosto che siano in accordo sull'ordine in cui gli eventi si svolgono. Nell'esempio *make*, i tempi assoluti in cui *input.c* e *input.o* sono stati creati sono irrilevanti; Ciò che conta invece è se *input.c* sia più vecchio o più recente di *input.o*.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Clock logici di Lamport

Per sincronizzare gli orologi logici, Lamport definì la relazione *happensbefore*. L'espressione $a \to b$ viene letta come "l'evento a si verifica prima dell'evento b" e indica che tutti i processi concordano sul fatto che l'evento a si verifica per primo, seguito dall'evento b. La relazione di precedenza può essere osservata direttamente in due situazioni:

1. Se a e b sono eventi nello stesso processo e a si verifica prima di b, allora $a \to b$ è un'espressione vera.
2. Se a è l'evento di un processo che invia un messaggio e b è l'evento di un altro processo che riceve il messaggio, allora $a \to b$ è un'espressione vera. Poiché un messaggio impiega un tempo finito e diverso da zero per arrivare a un messaggio, non può essere ricevuto prima che venga inviato o anche contemporaneamente al suo invio.

Considerando che la relazione definita come *happensbefore* è una relazione transitiva, se $a \to b$ e $b \to c$, allora $a \to c$.

Se due eventi, x e y, si verificano in processi separati che non scambiano messaggi (anche indirettamente attraverso terze parti), allora né $x \to y$ né $y \to x$ sono vere. Questi eventi sono concorrenti, il che significa semplicemente che è impossibile (o non necessario) determinare quando si sono verificati o quali si sono verificati per primi.

Abbiamo bisogno di un metodo per misurare una nozione di tempo tale che per ogni evento a, possiamo assegnargli un valore temporale $C(a)$ con cui tutti i processi concorrono.

Questi valori temporali devono soddisfare la seguente condizione: se $a \to b$, allora $C(a) < C(b)$.

Se a e b sono due eventi che si verificano all'interno dello stesso processo e a si verifica prima di b, allora $C(a) < C(b)$. Allo stesso modo, se a è l'invio di un messaggio da parte di un processo e b è la ricezione di quel messaggio da parte di un altro processo, allora $C(a)$ e $C(b)$ devono essere assegnati in modo tale che tutti concordino sui loro valori con $C(a) < C(b)$.

Inoltre, il tempo del clock C deve sempre aumentare e mai diminuire. I tempi possono essere regolati solo aggiungendo un valore positivo, mai sottraendone uno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Esaminiamo ora l'algoritmo proposto da Lamport per assegnare i tempi agli eventi. Si considerino i tre processi illustrati nella Figura 1.

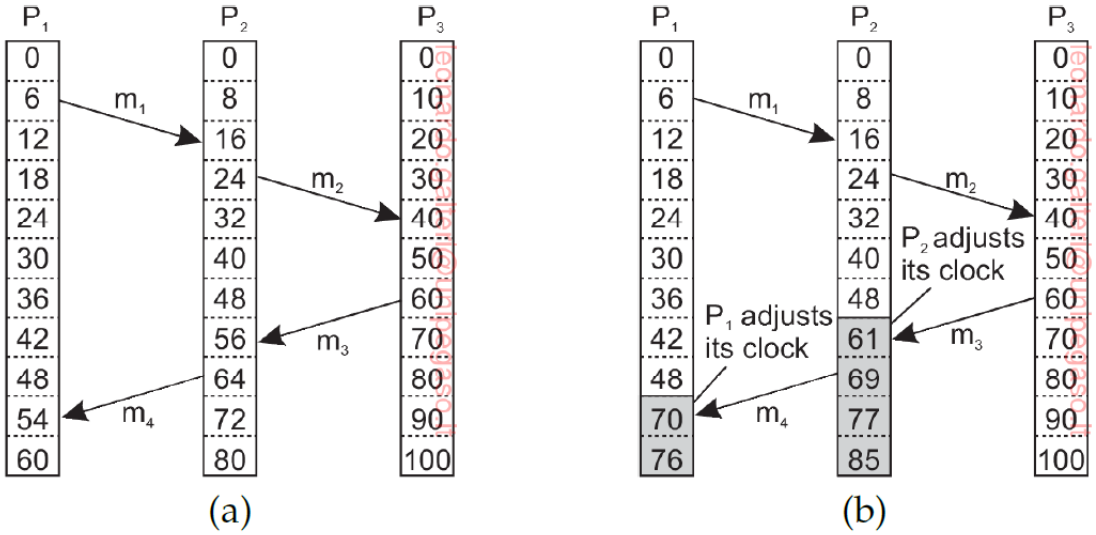


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

I processi vengono eseguiti su macchine separate, ognuna delle quali ha il proprio clock. Supponiamo che un clock sia implementato come contatore software: ogni T unità di tempo, il contatore viene incrementato di un valore specifico. Tuttavia, il valore di incremento di un orologio varia da processo a processo. Sei unità vengono aggiunte al clock nel processo P_1, otto unità nel processo P_2 e dieci unità nel processo P_3. (I clock logici di Lamport sono in realtà contatori di eventi, il che spiega perché i loro valori possono variare tra i processi.)

Al tempo 6, il processo P_1 invia il messaggio m_1 del processo P_2. Il tempo necessario per l'arrivo di questo messaggio dipende dal clock a cui si fa affidamento. In ogni caso, l'orologio P_2 del processo legge 16 quando arriva. Se il messaggio contiene l'ora di inizio, 6, il processo P_2 concluderà che il viaggio ha richiesto 10 tick. Certamente, questo valore è possibile. Utilizzando questa logica, il tempo di transito per il messaggio m_2 da P_2 a P_3 è di 16 tick, che è anch'esso un valore plausibile. Il messaggio m_3 parte dal processo P_3 a 60 e arriva al processo P_2 a 56. Il messaggio m_4 da P_2 a P_1 parte anche a 64 e arriva a 54. Chiaramente, questi valori sono impossibili. Questa particolare situazione deve essere evitata.

La soluzione proposta da Lamport deriva direttamente dalla relazione *happensbefore*. Dato che m_3 è partito a 60, deve arrivare a 61 o dopo. Di conseguenza, ogni messaggio include il tempo di invio in base al clock del mittente. Quando arriva un messaggio e il clock del destinatario indica un'ora precedente all'ora

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

in cui il messaggio è stato inviato, il destinatario anticipa il clock di un secondo. Nella figura 1, possiamo vedere che m_3 ha ora raggiunto 61. Allo stesso modo, m_4 raggiunge 70.

Specifichiamo questa procedura in modo più dettagliato, ricordando che è fondamentale distinguere tre livelli software distinti: la rete, un livello middleware e un livello di applicazione, come illustrato nella Figura 1. In genere, ciò che segue fa parte del livello middleware.

Ogni processo mantiene un contatore locale per implementare i clock logici di Lamport. I seguenti passaggi sono utilizzati per aggiornare questi contatori

1. P_i incrementa C_i: $C_i \leftarrow C_i + 1$ prima di eseguire un evento (ad esempio, inviare un messaggio sulla rete, recapitare un messaggio a un'applicazione o qualche altro evento interno).
2. Quando il processo P_i invia il messaggio *m* al processo P_j, imposta il timestamp del messaggio $ts(m)$ su C_i dopo aver eseguito il passaggio precedente.
3. Dopo aver ricevuto un messaggio *m*, il processo P_j regola il suo contatore locale come $C_j \leftarrow \max\{C_j, ts(m)\}$, quindi esegue il passaggio iniziale e invia il messaggio all'applicazione.

In determinate circostanze, è auspicabile che non si verifichino esattamente due eventi contemporaneamente. Per raggiungere questo obiettivo, si utilizza l'identificatore di processo univoco per rompere i legami e si fa utilizzo anche di tuple anziché dei valori del contatore. Ad esempio, un evento P_i del processo che si verifica al tempo 40 sarà contrassegnato come $\langle 40, i \rangle$. Se c'è anche un evento $\langle 40, j \rangle$ e $i < j$, allora si ha che $\langle 40, i \rangle < \langle 40, j \rangle$.

Si noti che assegnando all'evento il tempo $C(a) \leftarrow C_i(a)$ se a si è verificato durante un processo P_i al tempo $C_i(a)$, abbiamo un'implementazione distribuita del valore temporale globale che stavamo inizialmente cercando; in questo modo abbiamo creato un orologio logico.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Vector clock

Tutti gli eventi in un sistema distribuito sono completamente ordinati con la proprietà che se l'evento a si è verificato prima dell'evento b, allora anche l'evento a sarà posizionato in quell'ordine prima dell'evento b, cioè $C(a) < C(b)$.

Con gli orologi di Lamport, è impossibile determinare la relazione tra due eventi a e b semplicemente confrontando i loro rispettivi valori temporali $C(a)$ e $C(b)$. In altre parole, il fatto che $C(a) < C(b)$ non implica necessariamente che a si sia verificato prima di b.

Si considerino i messaggi inviati dai tre processi illustrati nella Figura 2 a titolo di esempio.

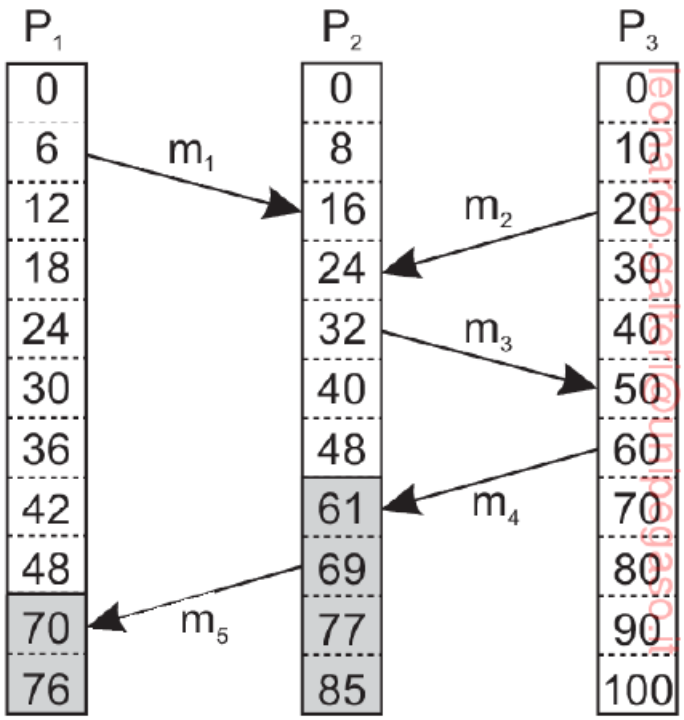


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

$T_{snd}(m_i)$ rappresenta l'ora logica in cui è stato inviato il messaggio mi e $T_{rcv}(m_i)$ rappresenta l'ora logica in cui è stato ricevuto. Per costruzione, sappiamo che $T_{snd}(m_i) < T_{rcv}(m_i)$ per ogni

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

messaggio. Tuttavia, cosa possiamo dedurre dal fatto che $T_{snd}(m_i) < T_{rcv}(m_j)$ per diversi messaggi m_i e m_j?

Nel caso in cui $m_i = m_1$ e $m_i = m_3$, sappiamo che questi valori corrispondono a eventi che si sono verificati durante il processo P_2, indicando che m_3 è stato inviato dopo m_1. Ciò potrebbe indicare che la trasmissione del messaggio m_3 dipendeva dal contenuto del messaggio m_1.

Inoltre, siamo consapevoli che $T_{rcv}(m_1) < T_{snd}(m_2)$. La figura 2 indica che la trasmissione di m_2 non ha nulla a che fare con la ricezione di m_1.

Il problema è che i clock di Lamport non sono in grado di catturare la causalità. Utilizzando clock vettoriali, la causalità viene catturata nella pratica. In effetti, il monitoraggio della causalità è semplice se diamo a ciascun evento un nome univoco, ad esempio un ID processo e un contatore che incrementa localmente: p_k è l'evento k-esimo che si è verificato nel processo P. Il problema si riduce quindi a tenere traccia delle storie causali. Ad esempio, se due eventi locali si verificassero al processo P, la storia causale $H(p_2)$ dell'evento p_2 sarebbe $\{p_1, p_2\}$.

Supponiamo ora che il processo P invii un messaggio al processo Q (che è un evento per P ed è quindi registrato come p_k per qualche k) e che al momento dell'arrivo (un evento per Q), la storia causale più recente di Q era q_1 (che è anche un evento per Q). Per tracciare la causalità, P invia anche la sua storia causale più recente (supponiamo che fosse $\{p_1, p_2\}$, con p_3 che esprime l'invio del messaggio) Q registra l'evento (q_2) all'arrivo e combina le due storie causali in una nuova: $\{p_1, p_2, p_3, q_1, q_2, q_3\}$.

Verificare se $H(p) \subset H(q)$ è un modo per determinare se un evento p causa un evento q. (cioè, dovrebbe essere un sottoinsieme appropriato). Infatti, con la nostra notazione, è sufficiente determinare se $p \in H(q)$, assumendo che q sia sempre l'evento locale finale in $H(q)$.

Il problema con le rappresentazioni delle storie causali è che sono inefficienti. Tuttavia, non è necessario registrare tutti gli eventi successivi dallo stesso processo poiché solo il più recente è sufficiente. Se assegniamo un indice a ciascun processo, possiamo rappresentare la storia causale come vettore, con la voce j-esima che rappresenta il numero di eventi che si sono verificati al processo P_i. Quindi, la causalità può essere catturata usando clock vettoriali, che sono costruiti consentendo a ciascun processo P_i di mantenere un vettore VC_i con le due proprietà elencate di seguito:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. $VC_i[i]$ rappresenta il numero di eventi avvenuti in P_i fino a quel momento. In altre parole, $VC_i[i]$ rappresenta il clock logico locale del processo P_i.
2. Se $VC_i[j] = k$, allora P_i è consapevole che k eventi si sono verificati in P_j. È quindi la conoscenza di P_i dell'orario locale in P_j.

La prima proprietà viene mantenuta incrementando $VC_i[i]$ ogni volta che si verifica un nuovo evento nel processo P_i. La seconda proprietà viene mantenuta allegando vettori ai messaggi trasmessi. In particolare, vengono eseguite le seguenti azioni:

1. P_i esegue $VC_i[i] \leftarrow VC_i[i] + 1$ prima di eseguire un evento (ad esempio, inviare un messaggio sulla rete, consegnare un messaggio a un'applicazione o qualche altro evento interno). Questo è lo stesso che notare un nuovo evento che si è verificato in P_i.
2. Quando il processo P_i invia il messaggio m al processo P_j, imposta il timestamp (vettoriale) $ts(m)$ su VC_i dopo aver eseguito il passaggio precedente (cioè, registra anche l'invio del messaggio come un evento che si verifica a P_i).
3. A seguito della ricezione di un messaggio m, il processo P_j regola il proprio vettore impostando $VC_j[k] \leftarrow max\{VC_j[k], ts(m)[k]\}$ per ogni k (che è equivalente ad unire le storie causali), dopodiché esegue il primo passo (registrazione della ricezione del messaggio) e quindi consegna il messaggio all'applicazione.

Se un evento a ha timestamp $ts(a)$, allora $ts(a)[i] - 1$ rappresenta il numero di eventi elaborati in P_i che precedono causalmente a. Di conseguenza, quando P_j riceve un messaggio m da P_i con timestamp $ts(m)$, è consapevole del numero di eventi in P_i che hanno preceduto causalmente la trasmissione di m. Ancora più importante, P_j viene anche informato del numero di eventi che si sono verificati in altri processi prima che P_i inviasse il messaggio m. In altre parole, il timestamp $ts(m)$ informa il ricevente del numero di eventi in altri processi che si sono verificati prima della trasmissione di m e da cui m può essere causalmente dipendente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Lamport L. Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM, 1978
- Raynal M. and Singhal M. Logical Time: Capturing Causality in Distributed Systems. Computer, 1996.
- Baquero C. and Preguica N. Why Logical Clocks Are Easy. Communications of the ACM, 2016.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).